



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Alan Rogelio Barrientos Ramírez

N° de Cuenta: 422022019

GRUPO DE LABORATORIO: 3

GRUPO DE TEORÍA: 6

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 12/09/2026

CALIFICACIÓN: _____

Cohete espacial y rombo de ocho pirámides

1 Ejecución de los ejercicios

Partí de practica3.cpp y de los shaders proporcionados en clase. Los dos ejercicios están en un solo main: con la tecla 1 se muestra el cohete y con la 2 el rombo. La tecla 3 permite revisar una pirámide por separado. Conservé las teclas E, R y T para girar los objetos y la proyección perspectiva de 60° con la relación de aspecto del ejemplo.

1.1 Cohete espacial

Para el cohete reutilicé las mallas del cilindro, cono, esfera, cubo y pirámide cuadrangular. El cuerpo central y los propulsores salen de cilindros; los conos forman las puntas, la tobera y las llamas. Las esferas forman la ventanilla, los cubos sujetan los propulsores y las pirámides funcionan como aletas.

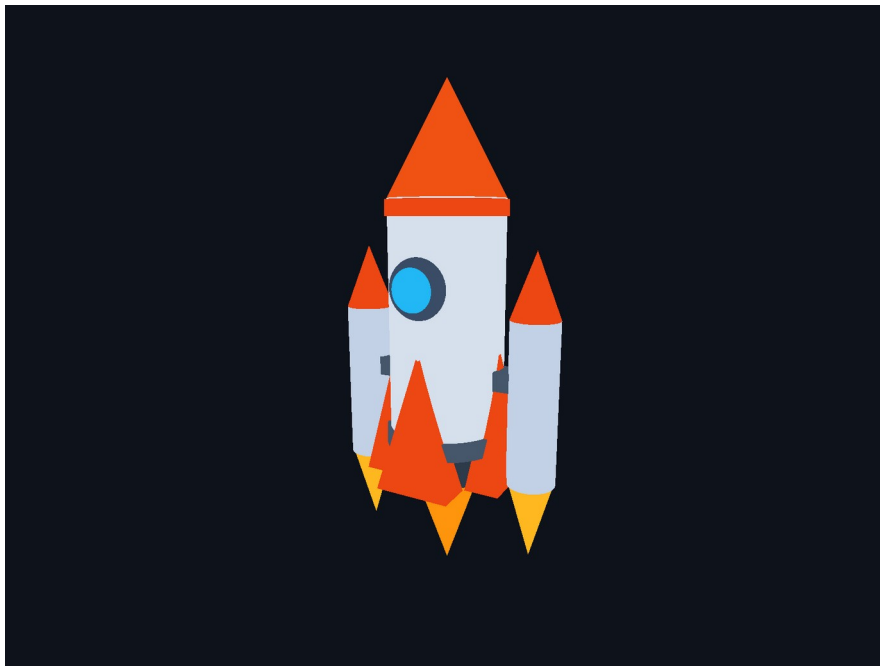


Figura 1. Ejecución del cohete construido con las cinco figuras solicitadas.

Malla	Instancias	Uso
Cilindro	5	Fuselaje, bandas y propulsores
Cono	7	Puntas, tobera y llamas
Esfera	2	Marco y cristal de la ventanilla
Cubo	2	Soportes laterales
Pirámide cuadrangular	4	Aletas

1.2 Bloques de dibujo y colores

Cada pieza empieza con la matriz identidad. Después apliqué las transformaciones del conjunto y las de la pieza, envié la matriz model y el color al shader, y dibujé la malla correspondiente. Repetí este bloque para las 20 instancias del cohete. La proyección no se cambió para acomodarlo: ajusté la escala y la posición de sus piezas.

```
practica3.cpp
431 // CILINDRO 01 - fuselaje central
432 model = glm::mat4(1.0);
433 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.80000f));
434 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
435 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
436 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
437 model = glm::translate(model, glm::vec3(0.00000f, 0.15000f, 0.00000f));
438 model = glm::scale(model, glm::vec3(0.48000f, 1.95000f, 0.48000f));
439 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
440 color = glm::vec3(0.84000f, 0.88000f, 0.93000f);
441 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
442 meshList[2]->RenderMesh();
```

Figura 2. Bloque del main que coloca y dibuja el cilindro central.

La pirámide de colores conserva las cinco posiciones y los seis triángulos de la pirámide cuadrangular. Cada cara tiene un RGB constante para que su color sea uniforme. Utilicé MeshColor para almacenar posición y color en cada vértice. Esta malla se dibuja con glDrawArrays, que utiliza el número de vértices del arreglo (Khronos Group, s. f.).

```
practica3.cpp
137 // Repito los vertices por cara para asignarles un color uniforme.
138 // Conservo las posiciones de la piramide y doy el mismo RGB a cada triangulo.
139 // Orden: frontal roja, izquierda verde, trasera amarilla,
140 // derecha magenta y los dos triangulos de la base azul.
141 GLfloat verticesPiramideColor[] = {
142     0.50000f, -0.50000f, 0.50000f, 1.00000f, 0.00000f, 0.00000f,
143     -0.50000f, -0.50000f, 0.50000f, 1.00000f, 0.00000f, 0.00000f,
144     0.00000f, 0.50000f, 0.00000f, 1.00000f, 0.00000f, 0.00000f,
```

Figura 3. Posiciones originales repetidas por cara y color rojo de la primera cara.

Cara de la malla original	Color	RGB
Frontal	Rojo	(1, 0, 0)
Izquierda	Verde	(0, 1, 0)
Trasera	Amarillo	(1, 1, 0)
Derecha	Magenta	(1, 0, 1)
Base cuadrada	Azul	(0, 0, 1)

1.3 Unión de las ocho pirámides

El segundo ejercicio tiene ocho instancias de la pirámide de colores: cuatro arriba y cuatro abajo. Las superiores comparten la punta $(0, 1, 0)$, y las inferiores la punta $(0, -1, 0)$. Sus bases cuadradas cubren el plano central $Y = 0$. El resultado es un rombo tridimensional cerrado, sin las puntas separadas que aparecían en los primeros acomodos.

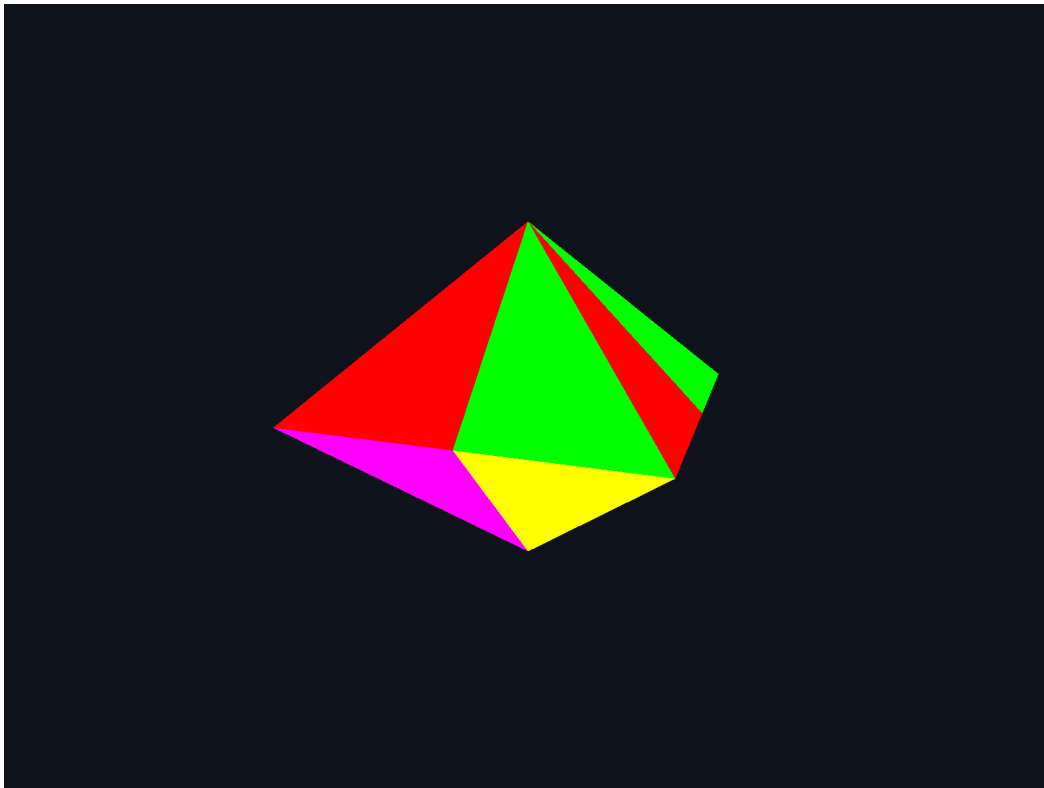


Figura 4. Rombo cerrado formado por las ocho pirámides.

Las bases azules quedan dentro de la unión. Para comprobarlas mantuve la vista individual, donde se puede girar una pirámide y revisar sus cinco caras.



Figura 5. Revisión de una pirámide por separado y de su base azul.

2 Problemas encontrados y solución

Lo más complicado fue unir las ocho pirámides. Al distribuirlas en dos niveles quedaban varias puntas separadas. Para cerrar la figura, incliné las piezas manteniendo sus bases cuadradas y acomodé cuatro arriba y cuatro abajo. Así, cada mitad comparte una sola punta. Tuve que cuidar el orden de las transformaciones, porque al cambiarlo también cambia la posición final de las piezas (GLM, s. f.).

Otra dificultad fue revisar la base azul, ya que queda dentro del rombo y no se distingue en la vista del conjunto. La escena de una pirámide individual permite girarla y comprobar las cinco caras. En el cohete, la rotación también ayuda a distinguir las aletas, los soportes y la profundidad de la ventanilla.

3 Conclusión

El cohete quedó construido con las cinco figuras solicitadas y el rombo con ocho pirámides de base cuadrada. Cerrar el rombo tomó más trabajo: repartir las piezas en dos niveles dejaba sus puntas separadas. La inclinación permitió unir las sin volver a definir la malla.

La vista individual me ayudó a comprobar los cinco colores de la pirámide, sobre todo la base azul que queda oculta en la unión. Al girar el conjunto pude revisar cómo encajaban las piezas. Lo más importante fue cuidar el orden de las transformaciones para colocar cada instancia sin cambiar los vértices de la malla.

4 Bibliografía

GLM. (s. f.). GLM_EXT_matrix_transform. <https://glm.g-truc.net/0.9.9/api/a00668.html>

Khronos Group. (s. f.). GLAPI/glDrawArrays. OpenGL Wiki.

<https://wikis.khronos.org/opengl/GLAPI/glDrawArrays>

Práctica 3: Modelado geométrico y cámara sintética. (2026). [Código C++ y shaders proporcionados en clase]. Facultad de Ingeniería, UNAM.